CS-107 Lecture 10
Von Neumann Architecture

Troy Astarte

Megan Venn-Wycherley

Recap: what each gate does

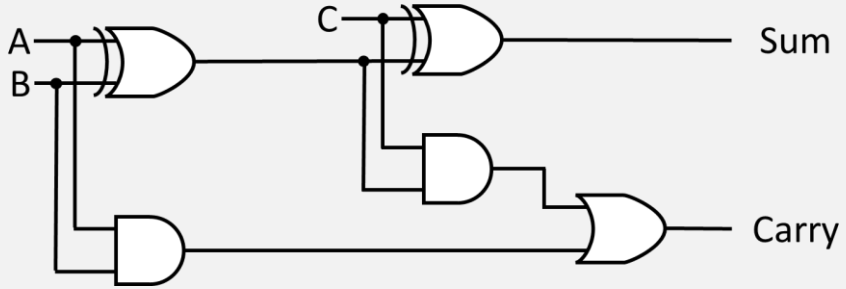
- **NOT:** one input, flipped - 1 becomes 0.
- **AND:** output is 1 only when both inputs are 1.
- **OR:** output is 1 when at least one input is 1.
- **XOR:** output is 1 only when the two inputs differ.
- **NAND:** an AND with the output inverted.
- **NOR:** an OR with the output inverted.

Recap: circuit equivalence

- **Two circuits are equivalent** when they give the same output for every combination of inputs.
- **The truth table is the proof:** $A \cdot (B + C)$ and $A \cdot B + A \cdot C$ match row for row.
- **Why it matters:** the simpler equivalent circuit needs fewer gates, so it is cheaper and faster.

Recap: adders

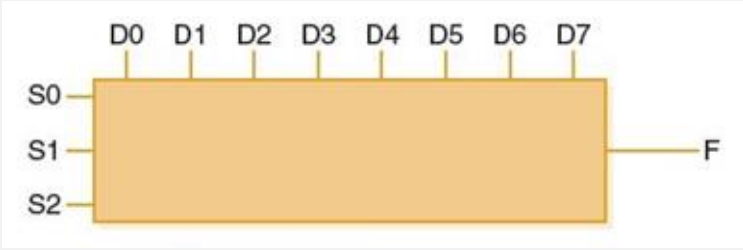
- **What it does:** an adder is a circuit built from ordinary gates that takes two binary numbers and produces their sum, plus a carry bit when a column overflows.
- **Half adder:** adds two single bits. The sum comes from an XOR gate (sum = A $\oplus$ B) and the carry from an AND gate (carry = A $\cdot$ B).
- **Full adder:** adds two bits together with a carry-in from the column to its right, so it works anywhere in a longer sum, not just the first column.



Recap: multiplexers

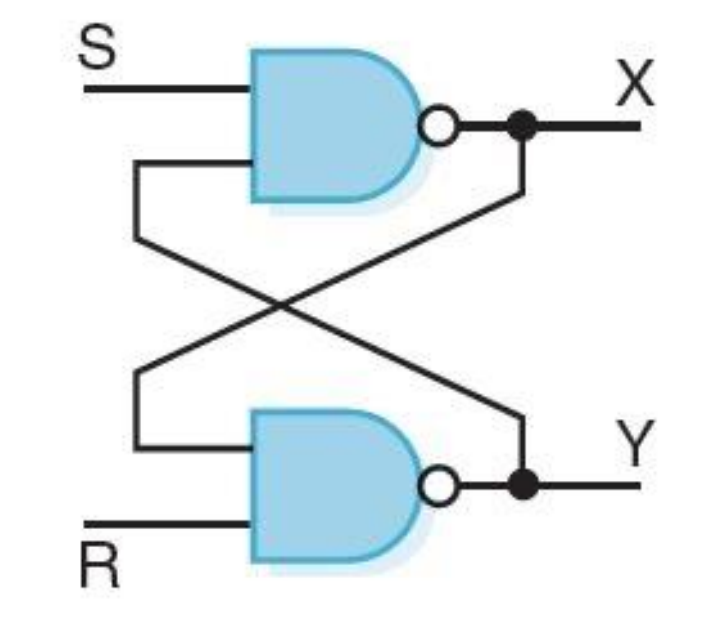
- **What it does:** a multiplexer is a switch built from gates. It reads its control signals, picks one of several input lines, and copies that line's value to a single output.
- **Control lines do the choosing:** with three control signals (S0, S1, S2) one of the eight data lines D0 to D7 is connected through to the output F.
- **n control lines select 2^n** inputs, so one control line chooses between two lines, two lines choose between four, and three choose between eight.

S0	S1	S2	F
0	0	0	D0
1	0	0	D1
0	1	0	D2
1	1	0	D3
0	0	1	D4
1	0	1	D5
0	1	1	D6
1	1	1	D7



Recap: latches store a bit

- **What it does:** a latch is memory built from gates. It stores one bit and keeps that value even after the inputs that set it stop changing.
- **How it is wired:** two NAND gates are cross-coupled, so each gate's output feeds back into the other gate's input and the pair settles into a stable state.
- **How you use it:** taking S to 0 sets the stored bit to 1, taking R to 0 resets it to 0, and holding both at 1 leaves the stored value alone.



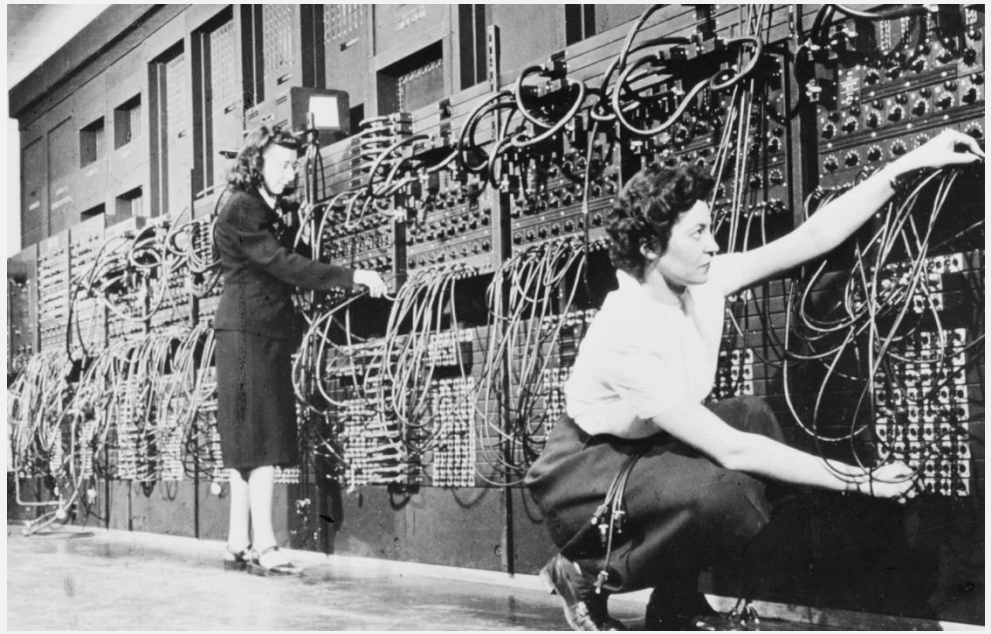
Goals for Today

- Where the stored-program idea came from, and who deserves the credit
- The five parts of the von Neumann architecture
- Memory, the ALU, the control unit and the buses between them
- The fetch-decode-execute cycle, step by step
- The von Neumann bottleneck, and how modern machines get around it

Before the Stored Program

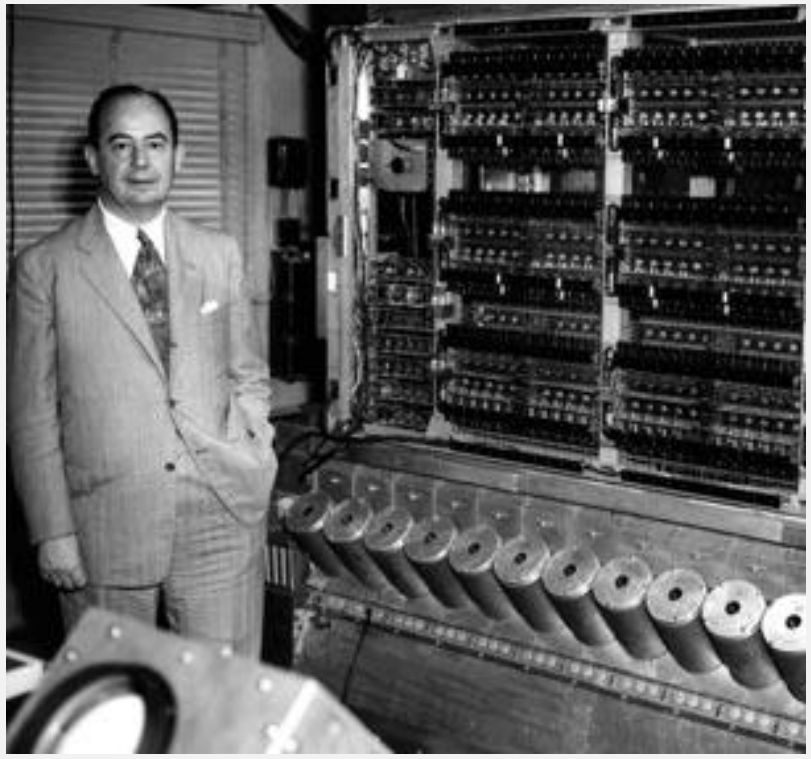
Programming used to mean rewiring

- ENIAC, switched on in 1945, was programmed by setting switches and plugging cables between panels.
- A new problem could take the operators days of physical work before a single answer came out.
- The women who did that work, among them Kay McNulty and Jean Bartik, planned the wiring by hand.



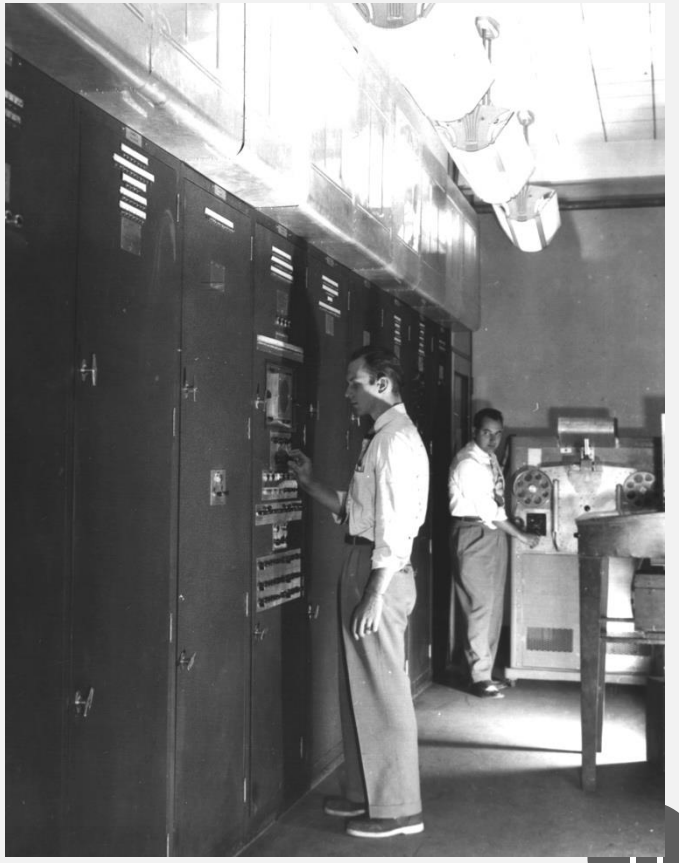
Who was John von Neumann?

- Hungarian-born mathematician, 1903 to 1957, at Princeton from 1930.
- He worked on quantum mechanics, game theory and the Manhattan Project - weapons work needed huge amounts of numerical calculation, which brought him to the ENIAC team in 1944.
- He then wrote up what the next machine, EDVAC, ought to look like.



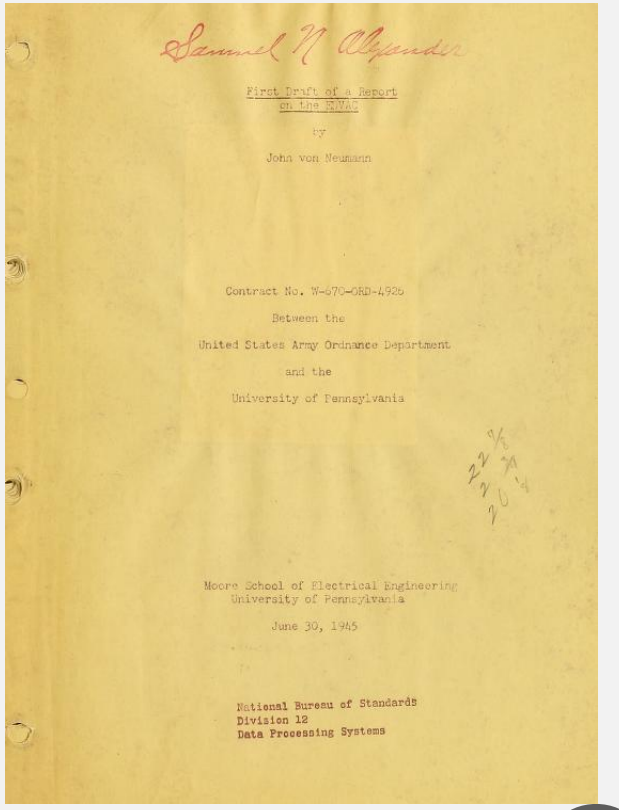
The 1945 EDVAC report

- "First Draft of a Report on the EDVAC" was circulated in June 1945.
- Its central idea: one memory holds instructions and data together, so a program is just numbers. It describes five parts: memory, arithmetic unit, control, input and output.



Whose idea was it?

- J. Presper Eckert and John Mauchly, who built ENIAC, had already proposed holding programs in memory.
- Alan Turing's 1936 universal machine treated program and data as the same kind of thing, on paper.
- Turing's 1945 report on the ACE described a stored-program computer in engineering detail.
- The draft went out with one name on the front (Von Nuemann), and the design has carried that name ever since.


Samuel N Alexander

First Draft of a Report
on the EDVAC

by

John von Neumann

Contract No. W-670-ORD-4926

Between the

United States Army Ordnance Department

and the

University of Pennsylvania

Moore School of Electrical Engineering
University of Pennsylvania

June 30, 1945

National Bureau of Standards
Division 12
Data Processing Systems

Stored-Program computers

Stored-program

- Programs can be **represented and stored as data**. There is no significant difference between instructions and data, therefore we can **store programs in our main data memory**.

The first stored-program machines

- Manchester Baby, 21 June 1948: the first program run from memory, in 32 words of storage.
- EDSAC at Cambridge, 1949: the first to do real work for other researchers.
- EDVAC itself did not run until 1951, six years after the report that described it!



The von Neumann Architecture

The von Neumann architecture

- A stored-program computer where instructions and data **cannot be fetched at the same time**, as they use a shared bus.
  - Remember the multiplexer last week?

- This causes a **bottleneck** on the processing as we need to perform fetching of instructions and data in **cycles.**
  - Understanding the situations that cause these bottlenecks will make us better programmers

The von Neumann architecture

- **Control unit**: Directs operations of the processor, using the instruction register and program counter.
- **Arithmetic/Logic unit**: Performs basic arithmetic and logical operations on processor's registers.
- **Memory**: Main data (including program) memory.
- **Bus**: Connection between CPU, memory, and peripherals.

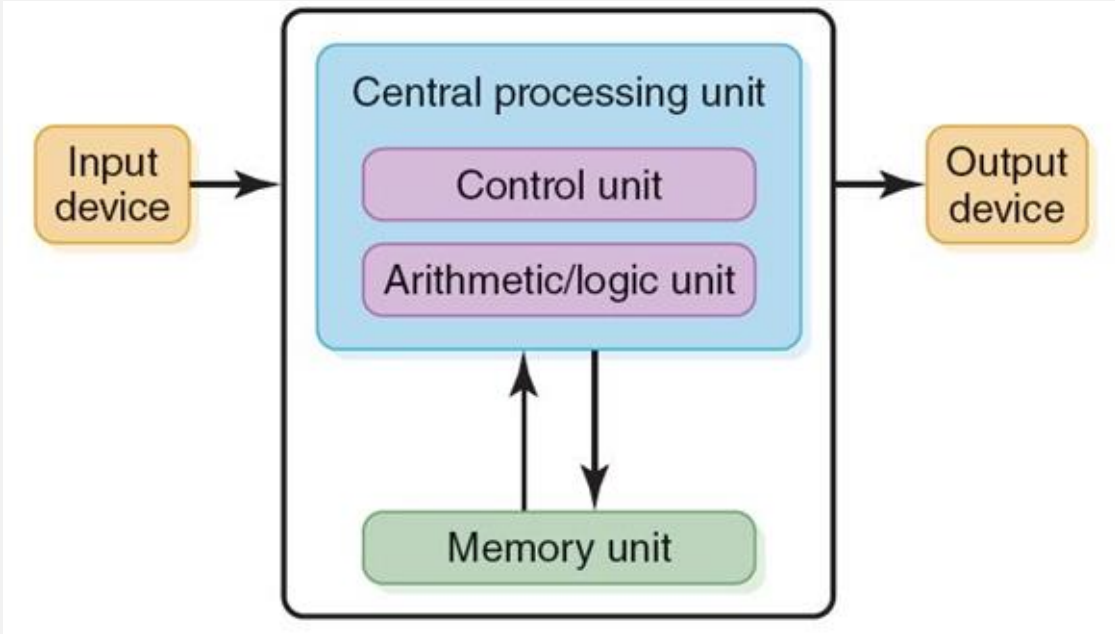


Image from Computer Science Illuminated. Jones and Bartlett

Control Unit

- This is the organising force in the computer. It provides **timing signals** and **control signals** which directs the ALU, memory, and I/O devices to respond to instruction.

- It also contains the **Instruction Register** and the **Program Counter.** These are both critical in the execution of instructions.

Control Unit

- **Instruction Register**
  - A register which **contains the instruction** that is currently **being executed**.
- **Program Counter**
  - A register which **contains the address of the next instruction** to be executed.
- These work together as part of the Fetch-Decode-Execute cycle to get the computer doing things.
  - More on this later

Arithmetic / Logic unit (ALU)

- Performs **basic arithmetic operations**, such as addition and multiplication.
- Performs **logical operations**, such as AND, OR, and NOT.
- Modern ALUs have a small number of small storage units called **registers**, which can be accessed faster than main memory.

Central Processing Unit (CPU)

- Commonly the control unit and ALU are contained within a singular component, named the **Central Processing Unit**

- The CPU also often contains additional registers and cache memory, with high communication speeds made possible by their close physical proximity.



Memory

- Collection of cells which **store information**. Each cell has a **unique physical address**.
- Most computers can address individual bytes of information.
- Often we want to read from / write to these memory locations.

Address	Content
0000 0000	1111 1101
0000 0001	1110 1001
0000 0010	0000 0000
0000 0011	0110 1010
…	…
1111 1101	1010 1111
1111 1110	0100 0110
1111 1111	1110 1101

Memory

- What does the 01000110 mean in memory address 11111110?
- It could be an instruction, a natural number, a signed integer, a character, part of an image…
- No way to really know without providing further context.

Address	Content
0000 0000	1111 1101
0000 0001	1110 1001
0000 0010	0000 0000
0000 0011	0110 1010
…	…
1111 1101	1010 1111
1111 1110	0100 0110
1111 1111	1110 1101

One byte, multiple meanings

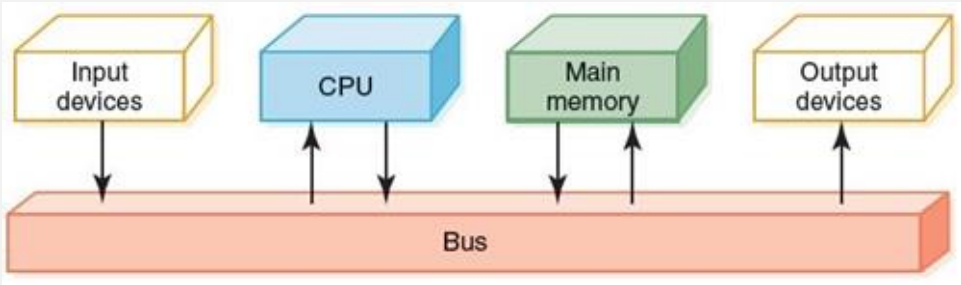
Memory address 11111110 holds the pattern 01000110. What is it?

1. Read as an unsigned binary number, it is 70.
2. Read as ASCII, it is the character F.
3. Read as an instruction, it might tell the CPU to load the next word into a register.
4. Read as part of an image, it is one mid-grey pixel.

Only the context that asked for it decides what this means!

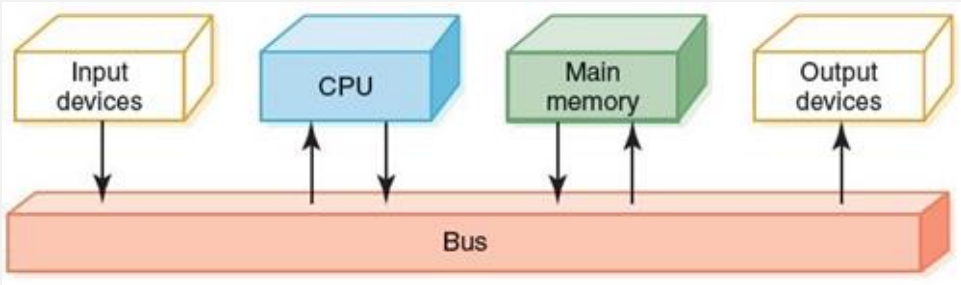
Bus (generic)

- Communication system that transfers data
- Can be between components inside a computer or between computers
- Includes the medium (wires, fibre, etc.) and protocols (rules)



Front-Side Bus

- This is **a dedicated bus** which connects the CPU, main memory, and other internal components (including other buses).



- This usually allows the communication of a **word-size** of information at any given time, controlled by the control unit with its **control signals** and **timing signals**.

The Harvard architecture

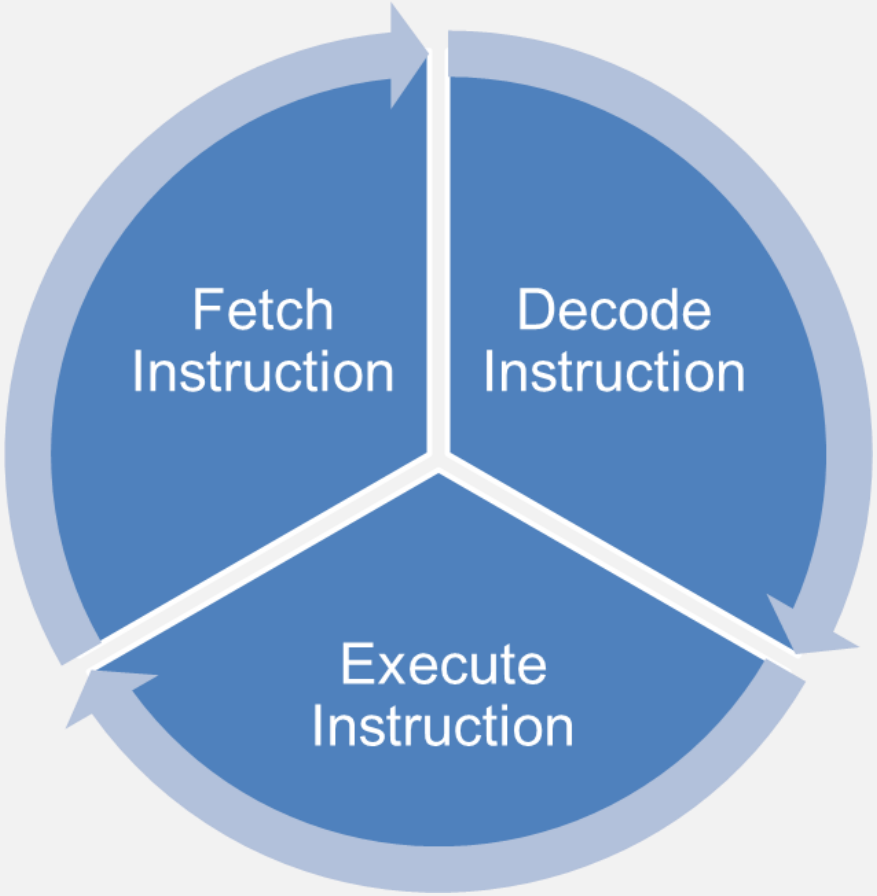
- The Harvard Mark I, running in 1944, read its instructions from punched tape and kept data separately.
- Separate instruction and data paths mean both can be fetched in the same moment.
- The cost is two memories to build and manage, and programs that cannot rewrite themselves.
- It lives on in microcontrollers, and in the split instruction and data caches inside modern CPUs.

The Fetch-Decode-Execute Cycle

The Fetch-Decode-Execute Cycle

- A cycle of CPU actions which allow the computer to perform operations. Sometimes also called the **Instruction Cycle:**
  - **Fetch** the next instruction
  - **Decode** the instruction
  - Get data from memory (if needed)
  - **Execute** the instruction

The Fetch-Decode-Execute Cycle



The Fetch-Decode-Execute Cycle

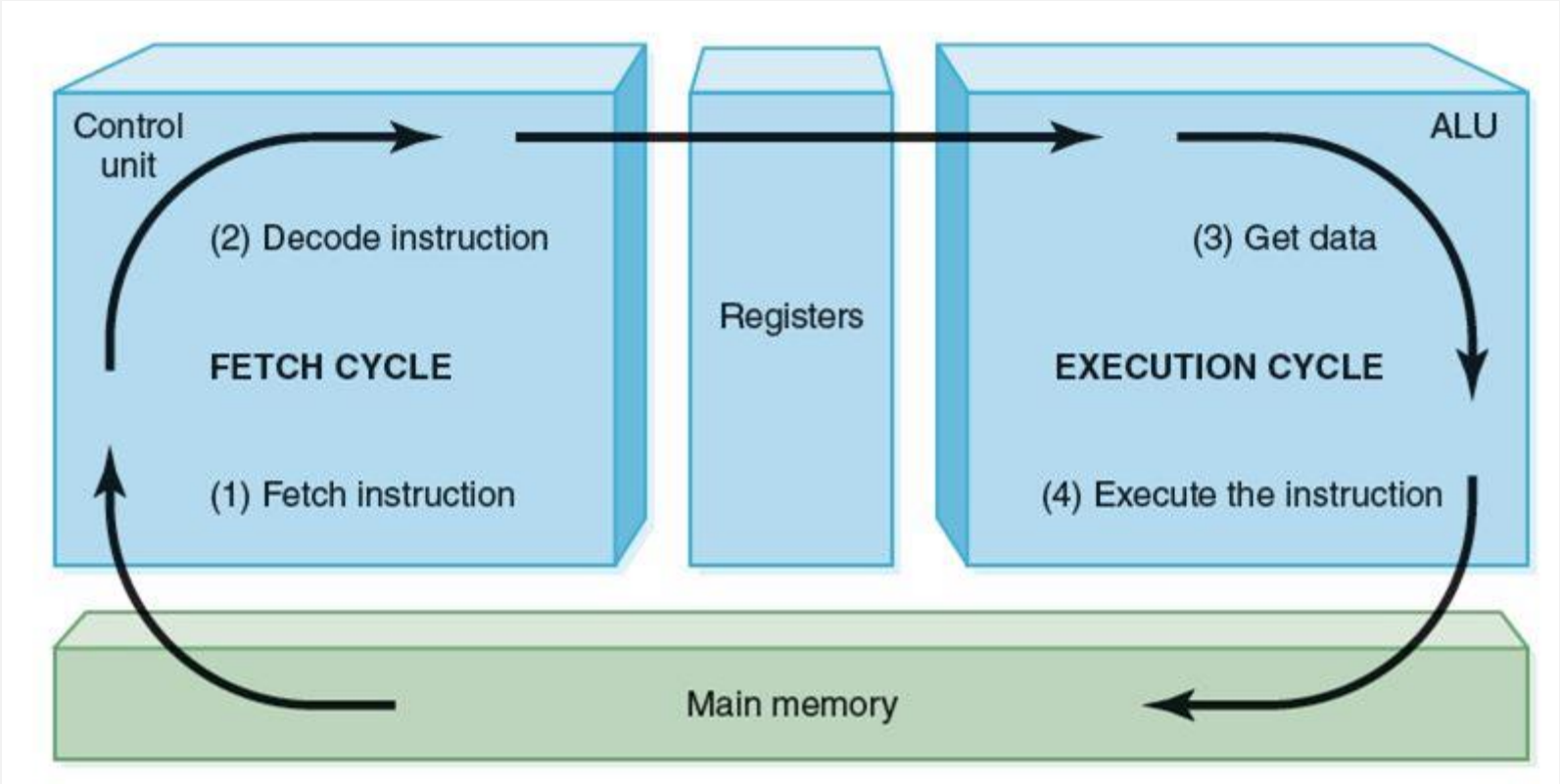
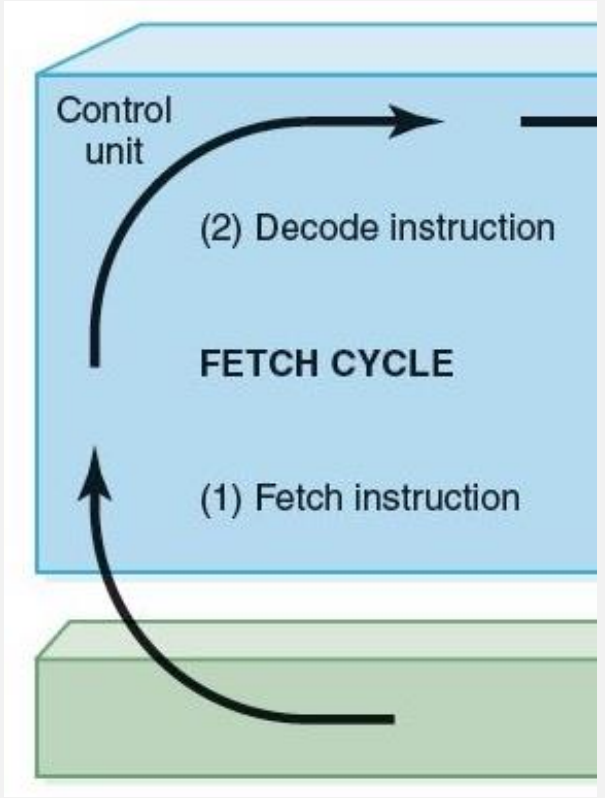


Image from Computer Science Illuminated. Jones and Bartlett

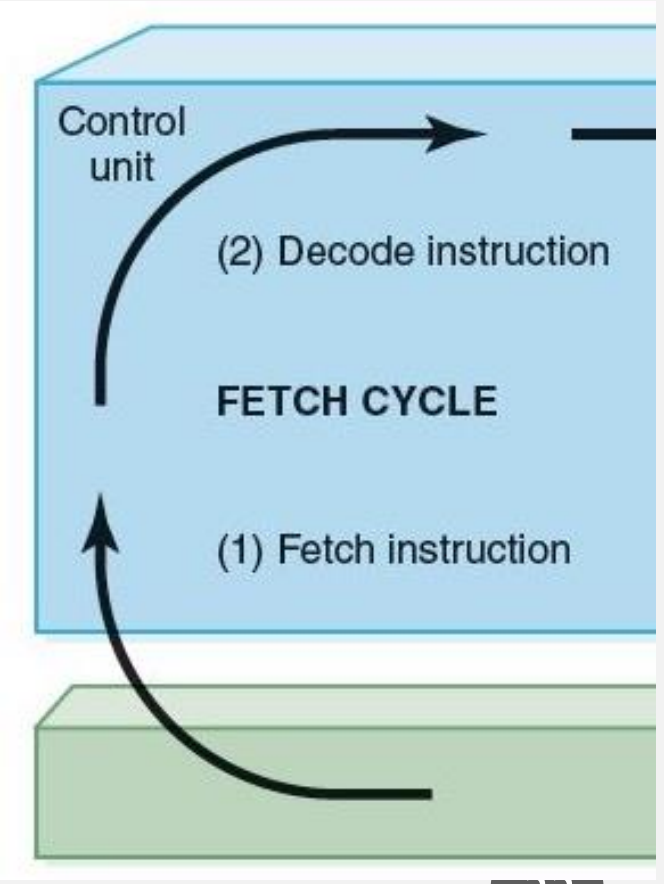
The Fetch-Decode-Execute Cycle

- **Fetch**: fetch the instruction from the memory location pointed to by the program counter.
  - **Memory location:** one numbered cell in memory, which should have data.
  - **Program counter:** the register holding the address of the next instruction to be fetched.
  - **Register:** a name store inside the processor, bult from latches, that holds the "word" the CPU is working on without having to go back out to memory.
- Place the instruction into the instruction register, then update the program counter to point at the next memory location.
  - So the program counter has already moved on while the instruction register still holds the current instruction.



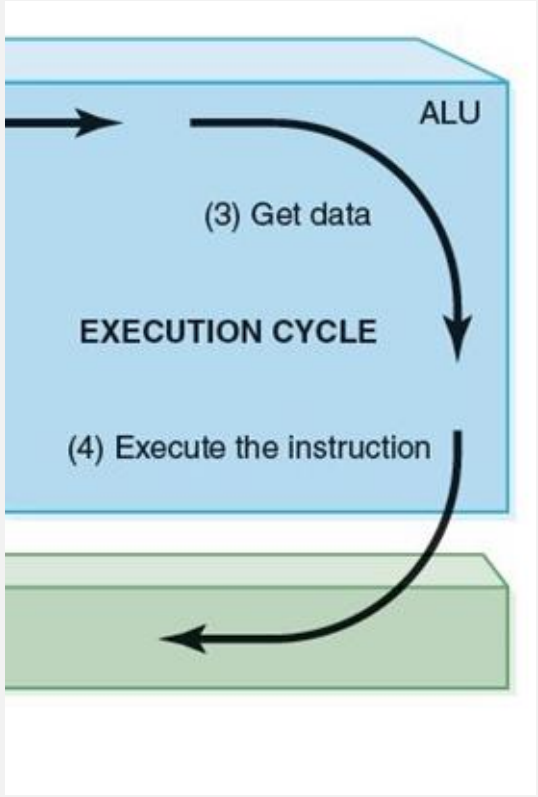
The Fetch-Decode-Execute Cycle

- **Decode**: The instruction (represented as a binary bit string) within instruction register is interpreted into an operation to perform.
  - **Program counter:** the register that holds the *next* instruction to be worked on.
  - **Instruction register:** the register holding the instruction being worked on right now.



The Fetch-Decode-Execute Cycle

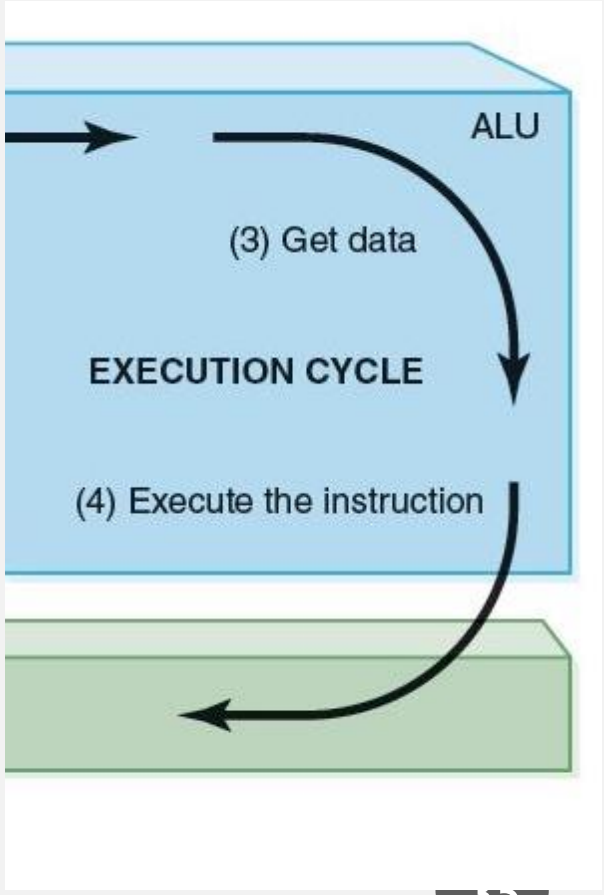
- **Read Memory**: If required, auxiliary data is fetched from main memory to be processed, and then placed into data registers.
  - Maybe you need to add two numbers held in memory. The instruction gives their addresses, not the numbers themselves, so each value has to be collected before the ALU can add anything.



The Fetch-Decode-Execute Cycle

- **Execute**: CPU's Control Unit passes the decoded instruction to functional units of the CPU, performing the actions required by the instruction.

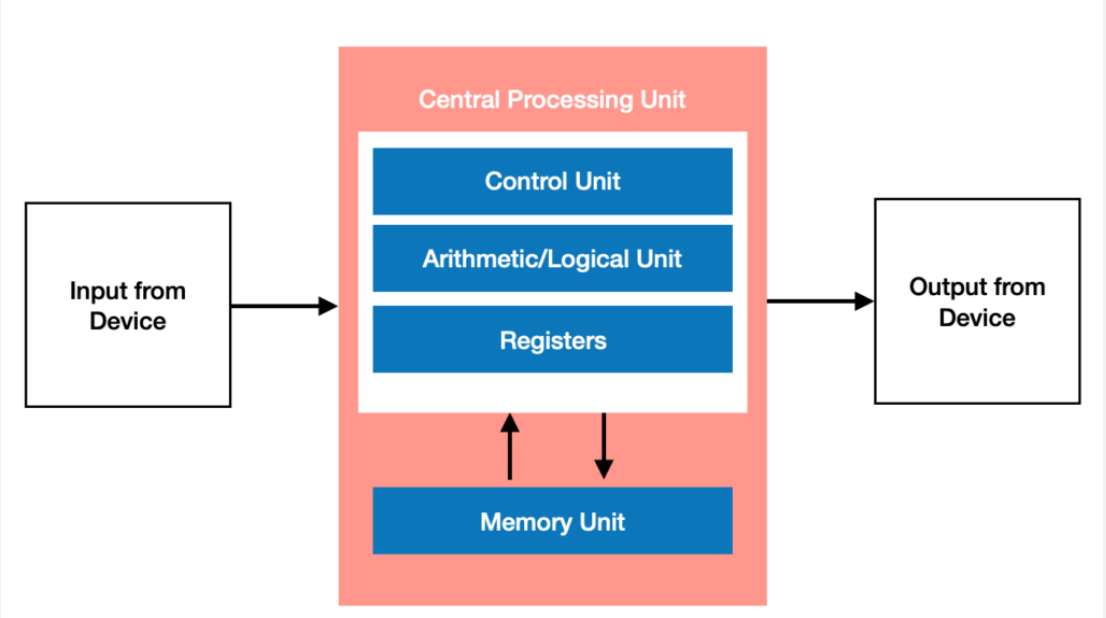
- The cycle then repeats, fetching the next instruction.



A Worked Example

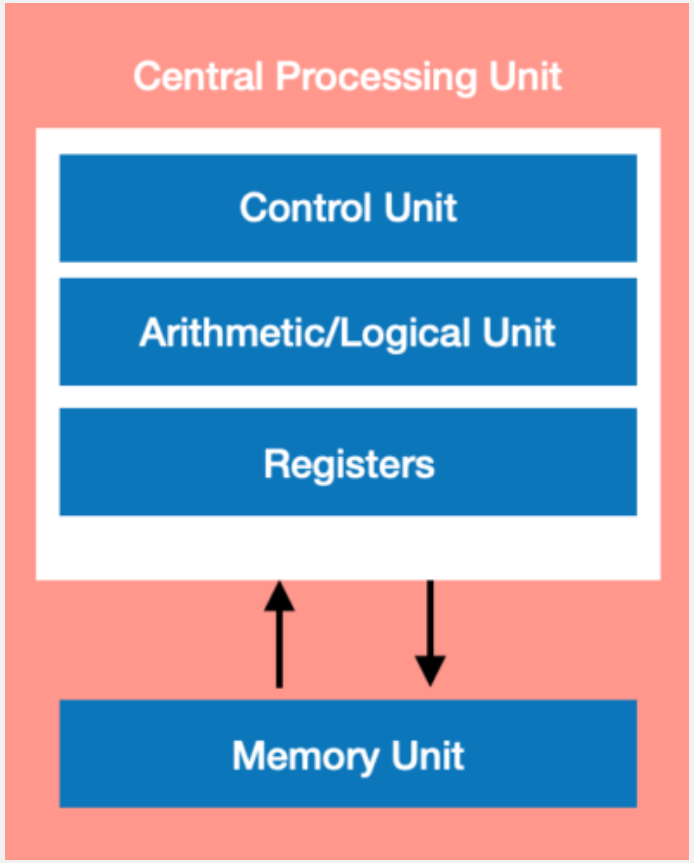
The example

- Let's add two numbers that are sitting in memory and put the answer back in memory – simple! But includes:
  - Instructions
  - Memory Location
  - Mathematical Operation
  - Write to Memory
  - Halt…



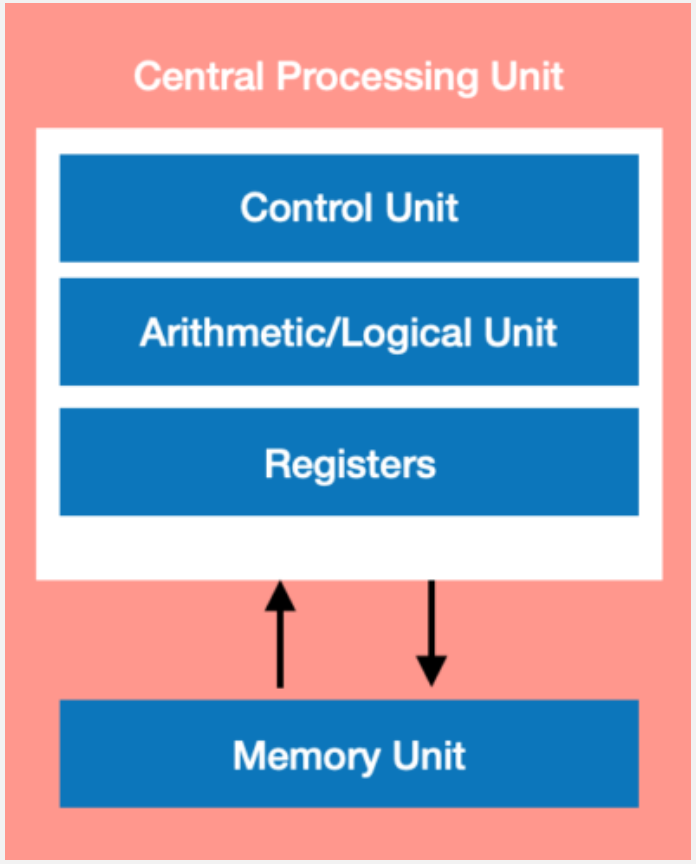
Step 1: fetch LOAD 200

- **Where next?** the program counter says the next instruction is at address 100, so the control unit asks memory what is stored there.
- **Along the bus:** memory sends the instruction back to the processor. (Trip 1 of 7.)
- **The processor now has it:** the instruction sits in the *instruction register*, and the *program counter* moves to 101, ready for next time.



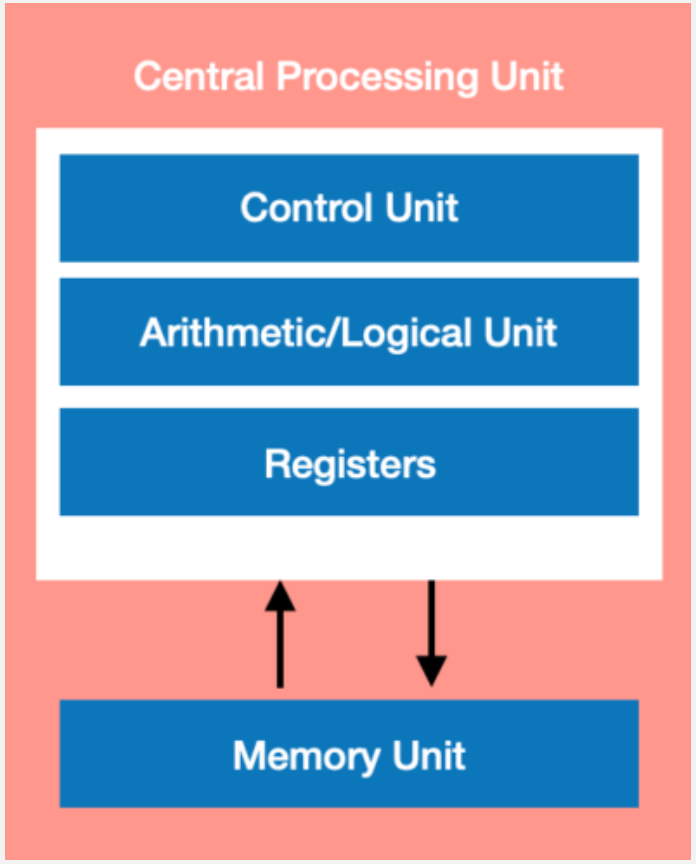
Step 2: decode LOAD 200

- **Reading the instruction:** the control unit splits the pattern into two parts, what to do and where to do it.
- **What to do:** load. Where: address 200.
- **What load means:** go and get the value kept at that address and hold it in the processor.
- **Nothing moves yet:** this step is only working out the plan, so the bus stays empty.



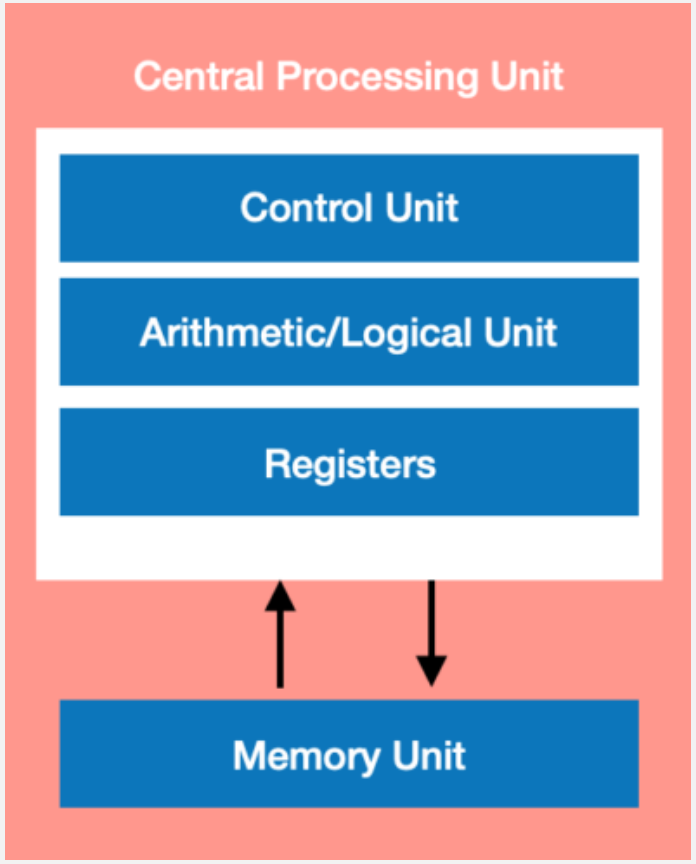
Step 3: execute LOAD 200

- **Carrying it out:** the control unit asks memory for the contents of address 200.
- **Along the bus:** memory sends back the pattern 00101101, which is the number 45. Trip 2 of 7.
- **Where it goes:** the value is held in the *accumulator*, the register the arithmetic and logic unit works with.
- **Counters:** next instruction 101, current instruction LOAD 200, accumulator 45.



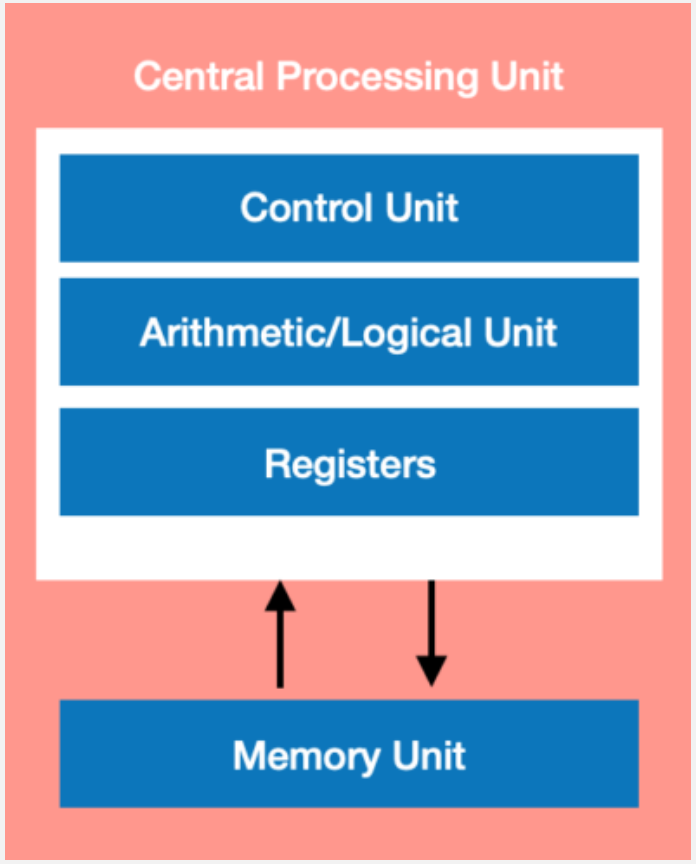
Step 4: fetch ADD 201

- **Back to the start of the cycle:** one instruction is finished, so the machine fetches the next one.
- **Where next?** the program counter says 101, so the control unit asks memory for the instruction stored there.
- **Along the bus:** ADD 201 comes back and replaces LOAD 200 in the instruction register. Trip 3 of 7.
- **Nothing is lost:** the program counter moves to 102, and the 45 stays in the accumulator.
- **Counters:** next instruction 102, current instruction ADD 201, accumulator 45.



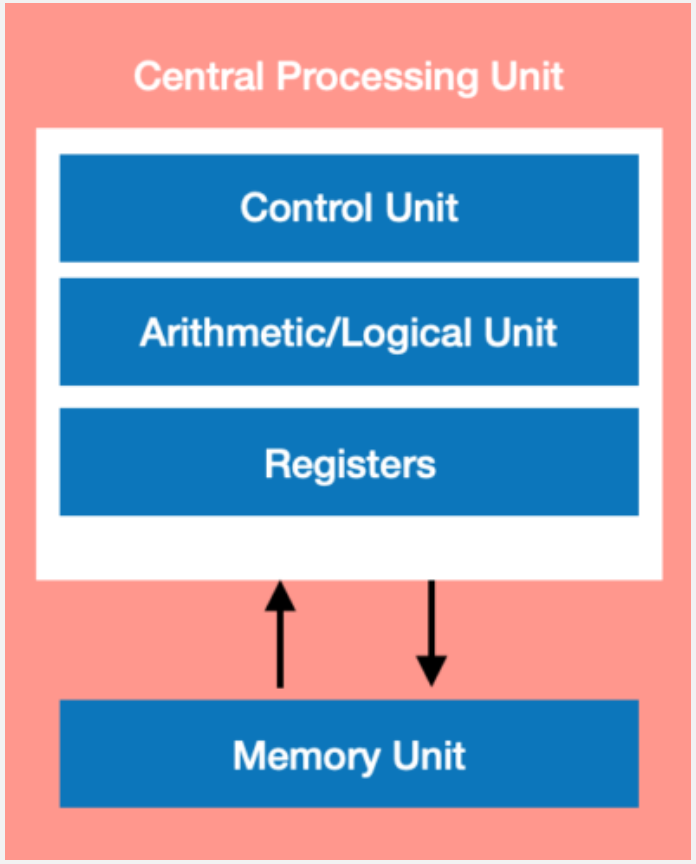
Step 5: decode and collect 201

- **Reading the instruction:** what to do is add. Where is address 201.
- **One number is already here:** 45 is in the accumulator from the last instruction.
- **The other is not:** the instruction gives an address, not a number, so it has to be collected.
- **Along the bus:** 00010011 arrives and waits in a data register. It is the number 19. Trip 4 of 7.
- **Counters:** next instruction 102, current instruction ADD 201, numbers ready 45 and 19.



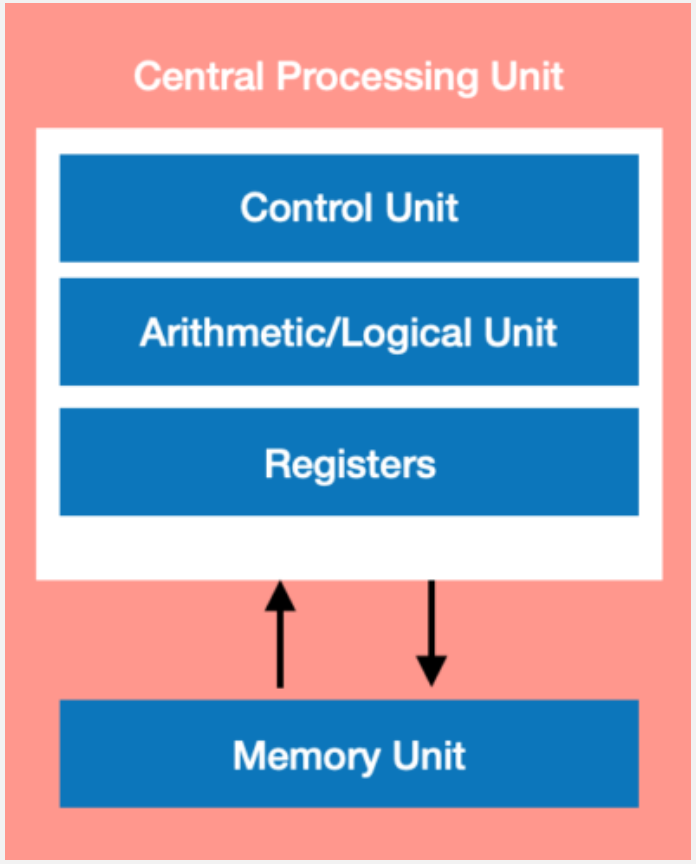
Step 6: execute the addition

- **Carrying it out:** the control unit hands both numbers to the arithmetic and logic unit and tells it to add.
- **Inside the ALU:** the 8-bit adder you saw last lecture works out 45 + 19 and produces 01000000, which is 64.
- **No bus trip needed:** both numbers were already inside the processor, so this step is fast.
- **The answer stays put for now:** 64 replaces 45 in the accumulator. Memory has not changed!
- **Counters:** next instruction 102, current instruction ADD 201, accumulator 64.
  - Nothing is properly stored yet, it's just sat waiting…



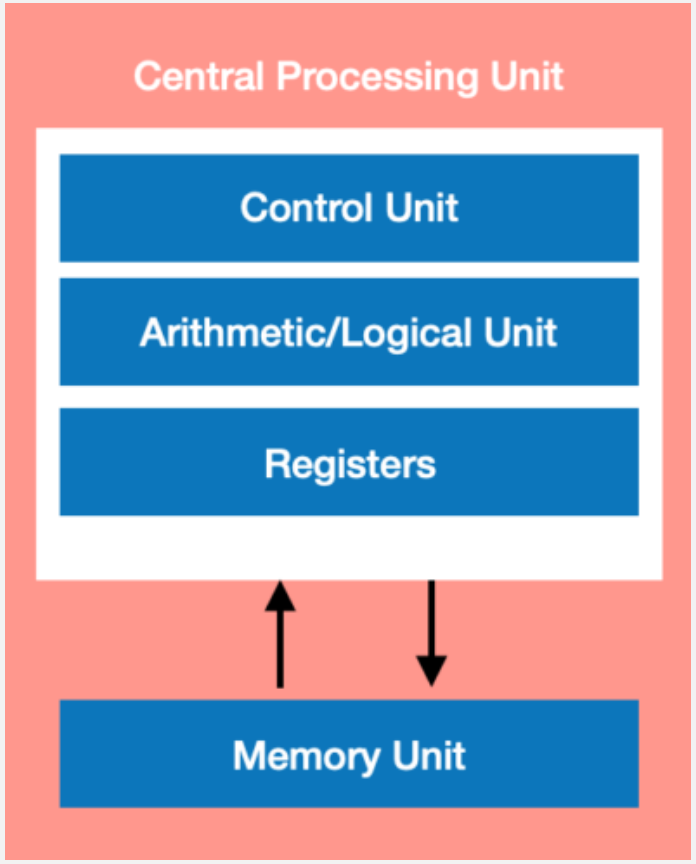
Step 7: store the result

- **Fetch:** the instruction at 102 comes back and the program counter moves to 103. Trip 5 of 7.
- **Read it:** what to do is *store*. Where is address *202*.
- **Carry it out:** the 64 in the accumulator travels out to memory and is written into address 202. Trip 6 of 7 (and the first one going outwards)
- **Counters:** next instruction 103, current instruction STORE 202, memory 202 now holds 64.



Step 8: halt

- **One last fetch:** the instruction at 103 comes back, and it means halt. Trip 7 of 7.
- **Stopping:** the control unit stops sending the timing signals that drive the cycle, so the machine stops.
- **The result:** 64 is sitting in memory at address 202, ready for whatever runs next (might not be needed).
- **Note:**
  - One addition took seven trips along the bus. Four carried instructions and three carried numbers.
  - The processor spent most of the time waiting for the bus rather than calculating.



Getting around the bottleneck

- **Cache:** a small, fast memory beside the CPU, so most fetches never reach main memory at all.
  - Not the same as a register: a register is a location the program writes into by name, while a cache keeps copies of memory the program asked for by address.
  - A register holds one word and there are a few dozen; a cache can hold thousands of words
  - A register is immediate, a cache takes a few cycles, main memory takes a few hundred.
- **Separate instruction and data caches:** Harvard ideas inside a von Neumann machine.
- **Pipelining:** fetch the next instruction while the current one is still executing.
- **Several cores:** each with its own caches, working at the same time.

From EDSAC to your laptop

- **EDSAC, 1949:** a few hundred instructions a second, about a thousand words of memory, a room-sized computer. **A laptop in 2026:** billions of instructions a second and billions of bytes of memory (definitely not room sized)
- **What changed:** speed, size, price, power and the number of cores.
- **What did not:** instructions and data share one memory, and every instruction is fetched, decoded, executed.
- **Not every machine works this way:**
  - A GPU runs thousands of small arithmetic units at once,
  - Quantum machines use qubits to hold many states at once and the answer comes from interference, not from a "stream of instructions".

Goals for Today

- Where the stored-program idea came from, and who deserves the credit
- The five parts of the von Neumann architecture
- Memory, the ALU, the control unit and the buses between them
- The fetch-decode-execute cycle, step by step
- The von Neumann bottleneck, and how modern machines get around it

CS-107 Lecture 11
Memory and Processing

Troy Astarte

Megan Venn-Wycherley

Goals for Today

- Reading a computer specification, and what the jargon names
- RAM and ROM, main memory and secondary storage
- How magnetic disks, optical disks and solid-state drives hold data
- Why access time matters: seek time, latency and transfer rate
- Parallel processing at bit, instruction, data and task level
- What all of it costs in a game running at 60 frames a second

Tech Jargon

- Let's imagine we want to buy a new computer.

- We open up a link for one that looks okay, and we see the following…



Tech Jargon

- **Operating system:** Windows 11
- **Processor:** Intel Core i5-14400, 10 cores, 2.5 GHz / 4.7 GHz, 20 MB cache
- **Graphics card:** NVIDIA GeForce RTX 5070
- **RAM:** 16 GB DDR4
- **Storage:** 1 TB SSD
- **Networking:** Gigabit Ethernet, dual-band Wi-Fi, Bluetooth
- **USB:** USB Type-C x 1, USB 3.0 x 4, USB 2.0 x 4

What does all this mean?!

Tech Jargon

- **Video connections:** DisplayPort x 3, HDMI x 1, on the graphics card
- **Audio connections:** 3.5 mm jack x 3
- **Disc drive:** none fitted
- **Memory card reader:** none fitted
- **Cooling:** air cooled, case fans front and rear
- **Mouse and keyboard:** optional extras, not included as standard
- **Case:** mid-tower, tempered glass side panel

Tech Jargon

- That's a lot of jargon (technical terms), and some of it may be more relevant to our needs than others.

- The key ones I personally often look for are:
  - Processor, RAM, Storage and Graphics Card
  - Ports (USB, video out etc.)

Tech Jargon (a closer look)

- **Processor:** The processing unit (usually a CPU), executes instructions and carries out low-level operations.

- Key details and rules of thumb:
  - Number of cores: how many things can it do in parallel, more is better
  - Speed: how fast can it process instructions, faster is better
  - Cache size: small amount of memory near the CPU for fast access

This is the control unit and the ALU. GHz counts fetch-decode-execute cycles per second, and cache is the shortcut past the bottlenecks on the bus

Tech Jargon (a closer look)

- **RAM:** Primary memory, allows you to have more data / larger programs in use at the same time.

- Key details and rules of thumb:
  - Size: how much information can I fit in memory, more is better
  - Speed: how fast can information be read/written, higher is better

RAM is the main memory in the diagram: the store the bus reaches on every fetch and every data read.

Tech Jargon (a closer look)

- **Storage:** How much information can you store in secondary memory?
- Key details and rules of thumb:
  - Size: how much information can I fit in memory, more is better
  - Speed: how fast can information be read/written, higher is better
  - Type: is it solid-state or hard disk? This affects things like speed and stability etc.

Types of Memory

Types of Memory

- **Random Access Memory (RAM)**: Each location can be accessed and changed.
- **Read Only Memory (ROM)**: Each location can be accessed, but not changed/modified.

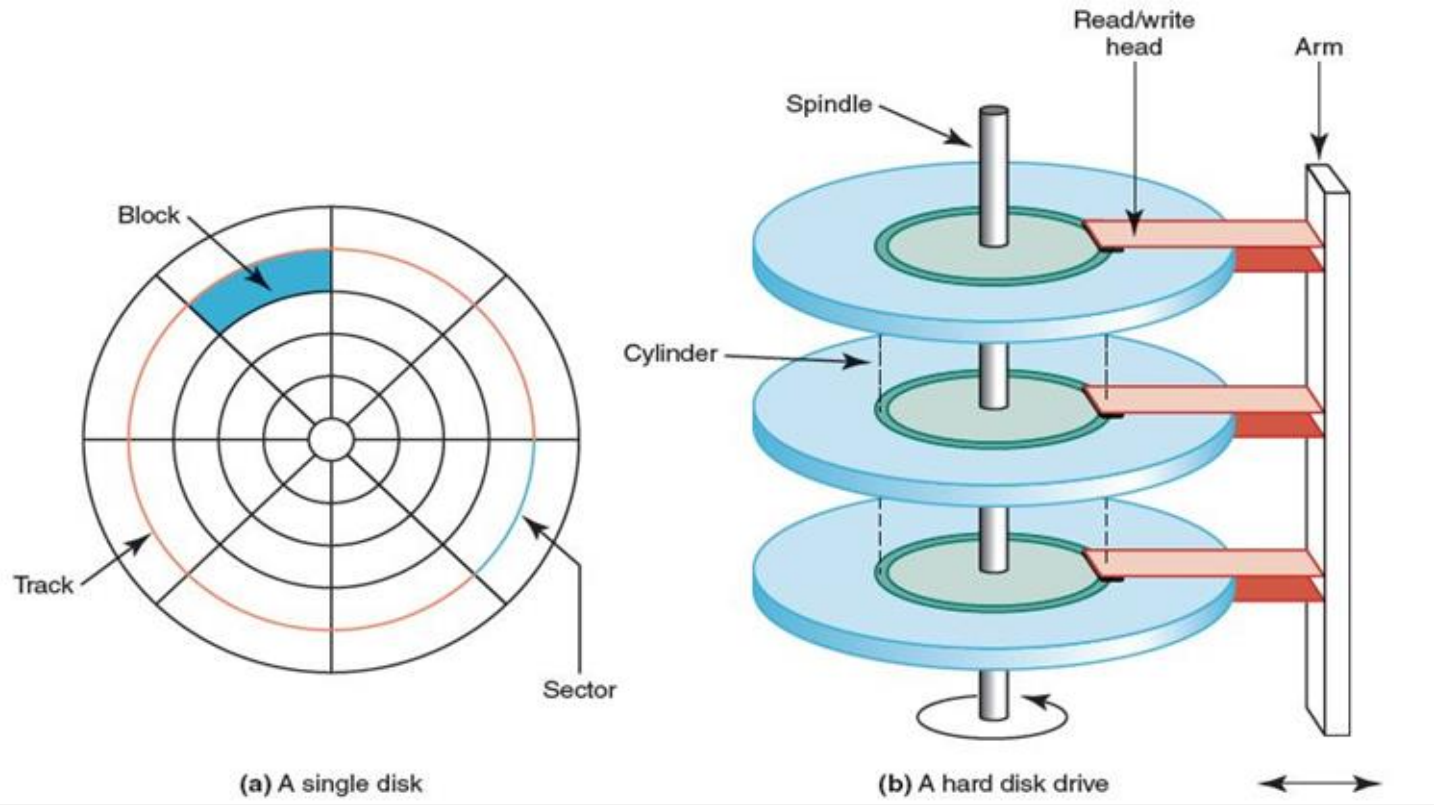
RAM is volatile, it stores the values with electrical voltage and when this voltage is removed the information is lost.

ROM is non volatile, the values are 'hard coded' into the address.

Types of Memory

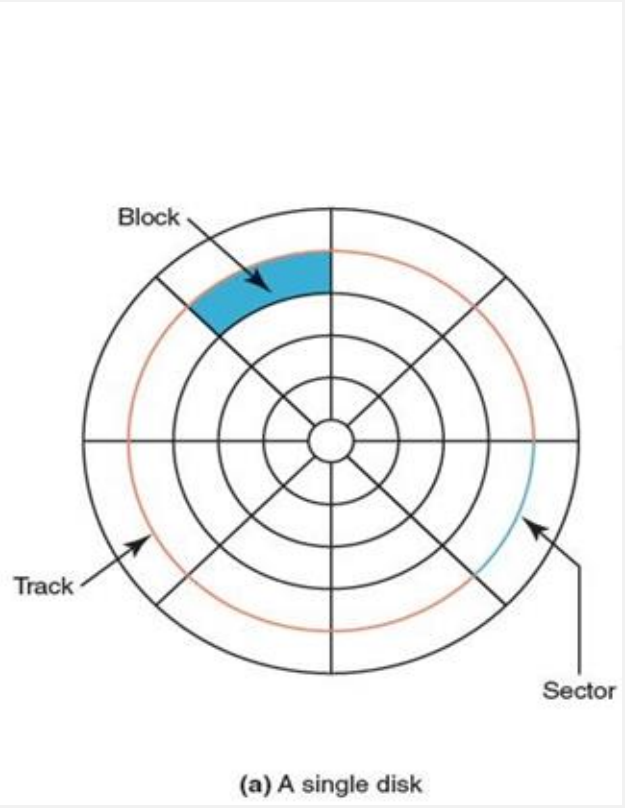
- **Main memory**: Storage location which can be **directly accessed by the CPU**, usually physically located on the computer to allow for extremely fast access to **active program and data** information.
- **Secondary memory**: Storage locations which are used for non-active information stored over longer periods of time. This can include devices such as **magnetic disks**, **optical disks**, and **solid state drives**.

Magnetic Disk Storage



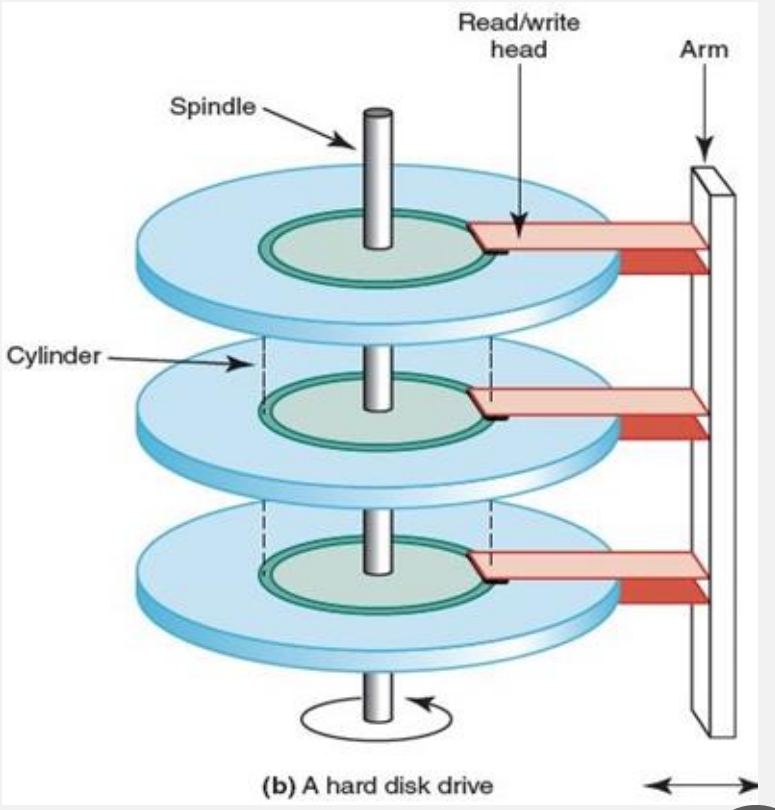
Magnetic Disk Layout

- **Track:** A physical, circular path on the disk surface. This is where the data is written/read.
- **Sector:** A sub-division of a track, storing a fixed amount of data.
- **Block:** Chunk of data on disk, identified by both track and sector. The minimal storage unit of a disk.



Magnetic Disk Layout

- **Cylinder:** A set of concentric tracks on all disks in a drive.
- **Spindle:** The central shaft that holds the disks and spins them all together at a constant speed.
- **Read/write head:** The sensor that flies just above a surface and magnetises it to write, or detects its magnetism to read. One head per surface.
- **Arm:** The assembly that carries the heads in and out



Magnetic Disk Access

- It takes time to access the information on the physical disk.
- **Seek Time**: time for read/write head to move over correct track.
- **Latency**: time for sector to be rotated under read/write head.
- **Access Time**: time for block to start being read (seek + latency)
- **Transfer Rate**: rate at which data moves from disk to memory.

Optical Disks

- **Compact disk (CD):** A disk that uses a laser to read information stored optically on a plastic-coated disk; data is evenly distributed around spiral track.
- **Digital versatile disk (DVD):** Used for storing audio and video.
- **Blu-ray:** Higher capacity DVD (using a blue light laser) allowing higher resolution video, etc.

Solid State Drives

- Uses **integrated circuits** to store data, rather than the magnetic or optical elements seen previously.
- Typically uses **flash memory** to store the information. This stores the value even when the power is removed, great for secondary storage.
- As it uses logical gates (transistors) to store data, there are no moving parts. This makes them much faster and quieter than mechanical disk drives, and provides impact resistance.

Processing

What we already know

- **One instruction at a time:** the machine we traced fetched, decoded and executed a single instruction before starting the next.
- **Everything shared one bus:** seven trips for one addition, and the processor waited through most of them.
- **We saw two ways out already:** keep copies close by in cache, and start the next instruction before the current one finishes…
  - This is still slow

Parallel Computing

- We discussed the Fetch-Decode-Execute cycle, and the limitations of the von Neumann bottleneck.
- In some scenarios it is beneficial to perform some operations **concurrently** (at the same time).
- This is called **parallelism**, and there are four common approaches to this task, depending on the application…

Parallel Computing

- Approaches to parallelism:
  - Bit-level
  - Instruction-level
  - Data-level
  - Task-level
- Modern computers often have multiple **cores** or processing units. Each processor is then able to carry out its own instructions.

Bit-level Parallelism

- Individual bits can be processed simultaneously
- For example,
  - If our computer's **word size** is 8 bits, it would require 2 operations to process a 16-bit value.
  - With a 16-bit word size we could do this task in a single operation.
  - Consider the 8-bit adder from last week. To add 16-bit numbers would require two applications of this adder. But if we had a 16-bit adder then we only need apply it once.

Note: What is "word size"?

- **Word size:** the number of bits the processor handles as one unit - the width of its registers, ALU and data bus. Individual bits can be processed simultaneously.
- For example,
  - If our computer's **word size** is 8 bits, it would require 2 operations to process a 16-bit value.
  - With a 16-bit word size we could do this task in a single operation.
  - Consider the 8-bit adder from last week. To add 16-bit numbers would require two applications of this adder. But if we had a 16-bit adder then we only need apply it once.
- **Why it matters:** word size sets how much data each instruction moves, the largest value the CPU handles natively, and how much memory it can address

Instruction-level Parallelism

- Some instructions can be executed independently
- This can take two main forms:
  - **Pipelining** is the concept of overlapping instructions which can be **carried out in series**. For example, in the Fetch-Decode-Execute cycle, can we start fetching the next instruction while our current one is decoding?
  - **Superscalar** is the concept of having multiple execution units which allows us to execute **multiple instructions per cycle**, increasing the **bandwidth** of our processor.

Data-level Parallelism

- Application of single instruction to many pieces of data simultaneously. **Single Instruction, Multiple Data** (SIMD)

- For example,
  - Calculating the average grades of all students at the same time.
  - Apply the same instructions to each student record in parallel.

Task-level Parallelism

- Entire tasks can be completed simultaneously, either with the same data or on different data.
- **Multiple instructions, multiple data** (MIMD)
- For example,
  - Different users carrying out different searches on Google

Bringing it all together





Control Resonant: the requirements

- Operating system: Windows 10 or 11, 64-bit
- Memory: **16 GB of RAM**
- Storage: 120 GB of **SSD** space
- Minimum, 1080p at 30 fps: **CPU** Core i5-8500 or Ryzen 5 3500; **GPU** GTX 1070 or RX 5600 XT with 6 GB of **VRAM**
- Recommended, 1440p at 60 fps: **CPU** Core i7-10700 or Ryzen 7 3700X; **GPU** RTX 3060 Ti or RX 6700 XT with 8 GB of **VRAM**

What one frame costs

- 30 frames per second allows 33 milliseconds per frame; 60 frames per second allows 16 milliseconds.
- A CPU core at 4.7 GHz completes about 75 million clock cycles in 16 milliseconds. Game logic, physics and AI share
- Remember that each instruction takes one or more cycles to fetch, decode and execute, so the number of cycles sets the number of instructions per frame.
- Instructions and data share one bus, so transfers to and from memory limit throughput: the von Neumann bottleneck.
- The GPU processes 3.7 million pixels per frame at 1440p, applying one instruction across thousands of units in parallel.

Reading the spec against our machine

Part	Our jargon machine	Control Resonant asks
CPU	Core i5-14400, 10 cores, 4.7 GHz	i5-8500 minimum; i7-10700 recommended
GPU	RTX 5070	GTX 1070 minimum; RTX 3080 at 4K; RTX 5080 for path tracing
RAM	16 GB DDR4	16 GB
Storage	1 TB SSD	120 GB SSD
OS	Windows 11	Windows 10 or 11, 64-bit

Architecture and Processing

- **Processing is the repeated execution of instructions.** The CPU fetches an instruction from memory, decodes it, executes it, then repeats: the fetch-decode-execute cycle.
- **Instructions and data are held in the same main memory** and travel over the same bus. That is the stored-program, or von Neumann, design.
- **Clock speed is cycles per second.** 4.7 GHz is 4.7 billion cycles per second, and an instruction takes one or more cycles, so clock speed caps instructions per second.
- **Word size and bus width set the work done per cycle:** the number of bits the processor operates on, and the number of bits moved between memory and processor.
- **Cores and GPUs add parallelism.** Each core runs its own instruction stream; a GPU applies one instruction across thousands of arithmetic units at once.
- **Access time rises with distance from the processor:** registers, cache, RAM, then SSD. Data must reach RAM before it can be processed, which is what a system requirement states for each component.

Goals for Today

- Reading a computer specification, and what the jargon names
- RAM and ROM, main memory and secondary storage
- How magnetic disks, optical disks and solid-state drives hold data
- Why access time matters: seek time, latency and transfer rate
- Parallel processing at bit, instruction, data and task level
- What all of it costs in a game running at 60 frames a second

CS-107 Lecture 11
Memory and Processing

Troy Astarte

Megan Venn-Wycherley